기민수

고객사 웹 서버 침해사고 두 건을 현장에서 처리했고, 사내 서버의 접근 경로와 백업을 다시 짰습니다.
퇴사 후에는 생성형 AI를 업무에 쓸 때 생기는 개인정보 유출과 진단 비용 문제를 풀려고 도구를
직접 만들었습니다.

이메일	전화	GitHub	웹 포트폴리오	영문 이름
kiminsoo365@gmail.com	010-4678-3603	github.com/ki0915	ki0915.github.io/Profile/docs	Ki Minsoo

키오스크 CMS를 만들면서 고객사 서버와 웹 서비스를 1년 11개월 동안 맡았습니다

미디어솔루션 연구개발팀에서 병역특례로 일했습니다. 키오스크 CMS를 만들었고, 납품한 뒤에는 대형병원, 대형 컨벤션센터, 5성급 호텔 두 곳의 서버와 웹 서비스를 맡았습니다. 고객사 웹 서버가 두 번 침해됐을 때 현장에 나가 대응했고, 병원 보안점검에서는 납품사 담당자로 수검에 대응했습니다.

사내 서버를 합칠 때 외부에서 바로 들어오던 원격 접속 경로를 없애고 백업을 여러 겹으로 나눴습니다. 퇴사 직후 회사에 랜섬웨어 사고가 났고, 회사는 이 백업으로 데이터 손실 없이 하루 만에 복구했습니다.

퇴사 후에는 생성형 AI 보안 도구를 만들었습니다. LLM으로 나가는 개인정보를 가리거나 막는 Mcp-Guard와 비용 상한을 둔 소스코드 진단 도구 ETK-Scanner는 혼자 만들어 공개했고, AI 에이전트 행동 감사 파일럿 EvidScope는 AI와 협업해 만들었습니다.

시기	내용
경력	
2023.04 – 2025.03	미디어솔루션 연구개발팀 사원 (병역특례)
학력	
2026.08	정보보호학 학사 (학점은행제)
2026.02	컴퓨터보안 전문학사 (명지전문대학)
자격	
2026.07	CLLMSP (LLM 보안)
2025.05	정보보안기사
2024.07	CISA 시험 합격 (Associate)
교육과 어학	
2025.07	TOEIC Speaking IH (140)
2024.11	KISA 케이쉴드 주니어 (80시간)

목차

	프로젝트	내용	쪽
실무, 미디어솔루션 2023.04 – 2025.03			
01	**침해사고 대응**	고객사 웹 서버 침해 2건, 데이터 검증 후 AWS 재구축	4
02	**병원 보안점검 대응**	권한 부여 순서의 경쟁 조건 수정, RBAC와 계정 정책 적용	6
03	**서버 통합과 백업 재설계**	외부 원격 접속 경로 제거, 백업을 여러 겹으로 분리	8
04	**Docker 디스크 고갈 장애**	미사용 이미지 정리를 cron에 등록해 재발 방지	10
AI 보안 프로젝트, 개인 2026			
05	**Mcp-Guard**	생성형 AI와 MCP 통신의 개인정보 유출 방지(DLP) 프록시, 공개 저장소	11
06	**ETK-Scanner**	비용 상한을 둔 LLM 기반 소스코드 취약점 진단(SAST), 공개 저장소	14
07	**EvidScope**	AI 기본법 기준 감사 증거 파일럿 (AI 협업)	16
실습과 저술			
08	**보안 운영 랩**	월간 보안 점검 비교, 디스크 장애 재현, 개인 랩 (AI 협업)	17
09	**모의해킹 실습과 Docker 보안 가이드**	TryHackMe 실습 보고서, OWASP와 CIS, NIST 기준 정리	19

두 번째 침해 뒤에는 검증한 데이터만 꺼내 AWS에 서비스를 새로 세웠습니다

고객사 웹 서버가 두 번 침해됐습니다. 1차는 KISA의 침해 통보로, 2차는 서비스가 멈췄다는 연락으로 알게 됐고, 두 번 모두 현장에 나가 대응했습니다.

1차 때는 시스템 접근과 관리자 계정부터 되찾았습니다. IIS 로그와 응용프로그램 풀에서 돌던 프로세스를 맞춰 보며 루트킷을 추적했고, 구형 미들웨어와 웹 필터의 취약점을 조치했습니다.

2차 때는 CPU와 메모리 과점유에서 크립토마이닝 의심 징후가 보였습니다. ASP.NET 4.7.1을 공개 CVE와 하나씩 대조해 취약한 구성을 찾았고, 재구축 결정을 도운 뒤 서비스를 EC2와 RDS로 옮겼습니다.

명세

기간	2023.04 – 2025.03 재직 중
구분	실무, 고객사 웹 서비스
역할	현장 대응, 원인 조사, 데이터 검증과 재구축, 고객 설명
기술	Windows Server, IIS, ASP.NET 4.7.1, MSSQL, AWS EC2, RDS, Security Group
설명	웹 포트폴리오에서 보기 ↗

결과 사고를 알게 된 날부터 복구를 마칠 때까지 4일이 걸렸습니다. 이관 뒤 제가 퇴사할 때까지 같은 원인의 사고는 다시 없었고, 파트너사와 클라우드 이관, 정기 서버 관리에 합의했습니다.

새로 세운 서버에는 운영 IP만 열어 두고 RDP는 막았습니다

1
현장 대응
두 번 모두 현장에 갔습니다. 1차 때는 시스템 접근과 관리자 계정부터 되찾았습니다.

2
원인 좁히기
1차는 IIS 로그와 실행 프로세스를, 2차는 ASP.NET 버전과 공개 CVE를 대조했습니다.

3
데이터만 꺼내기
MSSQL 데이터를 내부망 격리 PC로 옮겨 무결성을 확인했습니다.

4
새로 세우기
앱은 EC2, DB는 RDS로 옮겼습니다. Security Group은 운영 IP만 허용했습니다.

5
재발 방지 권고
VPN 도입, 프레임워크 업그레이드, AWS 이관을 권고안으로 남겼습니다.

1차와 2차

1차 때는 루트킷을 추적하고 구형 미들웨어와 웹 필터의 취약점을 조치했습니다. VPN 도입, 프레임워크 업그레이드, AWS 이관 권고안도 이때 썼습니다. 2차 때는 서비스가 이미 멈춰 있었고, 높은 CPU와 메모리 점유, 공격자가 설치한 가상화 환경 같은 이상 징후가 보였습니다. 이때 재구축 의사결정을 도왔고, 데이터 검증과 재구축은 제가 맡았습니다.

기술 담당자와 경영진에게 따로 설명했습니다

기술 담당자에게는 취약했던 구성과 조치 내용을, 경영진에게는 서비스 중단과 복구에 든 시간과 비용을 보여 줬습니다. .NET 버전별 CVE 수로 위험 감소율을 계산할 때는 AI를 활용해 추정치를 만들고 출처를 남겼습니다.

인증과 권한 구조를 병원 기준에 맞추면서 경쟁 조건 하나를 찾아 고쳤습니다

대형병원 키오스크 약 200대를 원격으로 관리하는 CMS를 혼자 설계하고 개발했습니다. 화면 미러링, 원격 명령, 예약 배포와 그룹 배포, 롤백을 넣어서 원격으로 풀 수 있는 장애는 현장(편도 5–20분)에 가지 않고 처리했습니다.

이 CMS가 병원 보안점검 대상이 되면서 권한 구분, 비밀번호 정책, 인증 처리를 병원 기준에 맞춰야 했습니다. 인증과 권한 구조를 고치는 과정에서 Redis 세션 검증이 끝나기 전에 임시 권한이 먼저 붙는 경쟁 조건을 찾아 수정했고, RBAC와 계정 정책을 넣었습니다.

명세	
기간	2023.04 – 2025.03 재직 중
구분	실무, 보안점검 수검 대응 (납품사)
역할	CMS 설계와 개발, 인증과 권한 구조 수정, 점검 대응
기술	TypeScript, Express, Angular, MariaDB, Redis, JWT, bcrypt, WebSocket, NICE CPClient, Electron
저장소	회사 소스라 공개하지 않음
설명	웹 포트폴리오에서 보기 ↗

결과 납품한 CMS 전체가 병원 보안점검을 통과했습니다.

권한은 Redis 세션 검증이 끝난 다음에만 붙게 했습니다

수정 전 로그인 요청 → **임시 권한 부여** → Redis 세션 검증

수정 후 로그인 요청 → Redis 세션 검증 → **권한 부여**

요청 타이밍에 따라 아직 검증되지 않은 세션이 잠깐 권한을 쥘 수 있는 구조였습니다. 권한 부여를 검증 뒤로 옮겨 검증된 세션만 권한을 받게 했습니다.

망분리된 병원 내부망에서는 NICE CPClient를 Electron 앱, 그리고 IIS와 .NET으로 만든 로컬 인증 서버와 콜백으로 이어 본인인증을 붙였습니다. Windows 업데이트 뒤 인증이 깨졌을 때는 원인을 찾아 복구하고 절차를 문서로 남겼습니다.

점검 항목	적용한 통제
권한 분리	관리자와 운영자 기능을 RBAC로 분리
토큰	서버 개인키로 서명한 JWT
비밀번호 저장	bcrypt 해시
비밀번호 정책	90일마다 변경, 이전 비밀번호 재사용 금지
로그인 실패	실패 횟수를 넘으면 계정 잠금
외부 통신	본인인증에 필요한 경로만 열고, 인증 이력은 한곳에서 관리

퇴사 직후 난 랜섬웨어 사고를 회사는 제가 나눠 둔 백업으로 하루 만에 복구했습니다

사내 서버는 노후 장비 7대였고, 웹 서버와 DB 서버에 외부에서 바로 원격 접속할 수 있었습니다. 서버마다 하는 일과 외부 노출을 확인한 뒤 Windows Server 2022와 Hyper-V로 합쳤습니다. 인바운드는 기본 차단으로 바꾸고, 아웃바운드는 필요한 것만 열었습니다.

Hyper-V 스냅샷 하나뿐이던 백업은 변경이 제한된 별도 볼륨, AWS 서버 이미지, MSSQL 복제로 나눴고, API 서버는 이중화했습니다. 백업에 접근하는 경로는 사내 게이트웨이 IP 중심으로 묶었고 복구 절차서를 썼습니다.

명세	
기간	2023.04 – 2025.03 재직 중
구분	실무, 사내 인프라
역할	점검, 통합과 이관, 백업과 이중화 설계, 복구 절차서
기술	Windows Server 2022, Hyper-V, IIS, 방화벽 정책, MSSQL 복제, AWS 서버 이미지
설명	웹 포트폴리오에서 보기 ↗

결과 2025년 4월, 제가 퇴사한 직후 회사에 랜섬웨어 사고가 났습니다. 회사는 이 백업으로 데이터 손실 없이 하루 만에 서비스를 복구했습니다. 복구 작업은 회사가 했고, 저는 백업 체계의 설계와 구현을 맡았습니다.

웹과 DB 서버로 바로 들어오던 원격 접속 경로부터 없앴습니다

영역	전	후
서버	노후 서버 7대	Windows Server 2022와 Hyper-V로 통합
접근 경로	웹과 DB 서버에 외부에서 직접 원격 접속	인바운드 기본 차단, 내부망을 거쳐서만 접속
복구 수단	Hyper-V 스냅샷에만 의존	변경 제한 백업 볼륨, AWS 서버 이미지, MSSQL 복제, API 이중화 서버
절차	–	복구 절차서, 백업 접근은 사내 게이트웨이 IP 중심으로 제한
구형 서버	Windows XP에서 돌던 출결 서버	새로 개발해 IIS와 인트라넷 DB에 연결

팀원들이 번거로워한 일

사내 PC 이미지를 주기적으로 갱신하는 일을 팀원들이 번거로워했습니다. 백업이 왜 필요한지 설명하고, 그 이미지로 테스트 서버를 빨리 만들 수 있다는 이점을 들어 설득했습니다. 번거롭지 않도록 절차도 문서로 남겼습니다.

배포 때마다 남은 Docker 이미지와 중지된 컨테이너가 호텔 서버 디스크를 채웠습니다

5성급 호텔 고객사 서버의 디스크가 가득 차 서비스가 멈췄습니다. 미사용 이미지와 중지 컨테이너를 원인으로 확인해 정리하고 서비스를 다시 올렸습니다. 같은 정리를 cron에 등록해 주기적으로 돌게 했습니다.

MariaDB 로그가 쌓여 생긴 장애도 해결했습니다. 그 뒤 로그와 이미지 정리 절차, 서버마다 확인할 항목을 정한 점검 기준을 문서로 만들었고, 지원이 끝난 Ubuntu 18.04는 22.04로 올렸습니다. Grafana와 Prometheus로 CPU, 메모리, 스왑, 컨테이너 자원을 보며 임계치에 닿기 전에 조치했습니다.

퇴사 후 이 장애를 개인 랩에서 다시 만들어 봤습니다. 08번에 있습니다.

명세	
기간	2023.04 – 2025.03 재직 중
구분	실무, 5성급 호텔 고객사
역할	서버와 DB 유지보수, 장애 대응, 운영 문서
기술	Ubuntu, Docker, cron, MariaDB, SSL VPN, Grafana, Prometheus
설명	웹 포트폴리오에서 보기 ↗

결과 cron 정리를 등록한 뒤로 제가 있는 동안 같은 원인의 디스크 장애는 다시 나지 않았습니다.

일반 HTTP 프록시가 보지 못하는 MCP stdio 통신까지 개인정보를 검사합니다

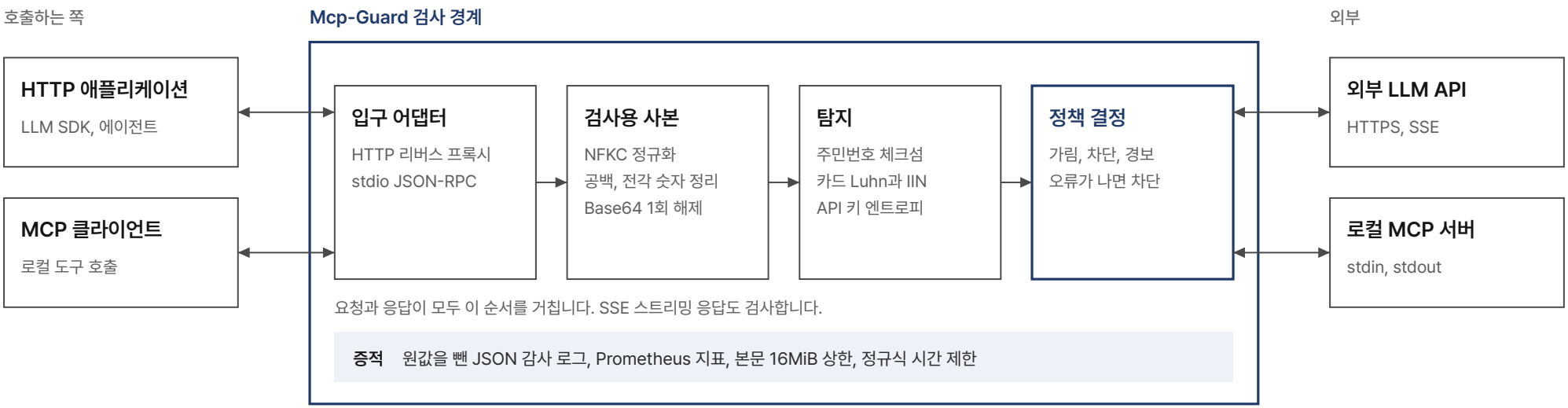
LLM API나 MCP 도구를 쓰다 보면 주민등록번호, 카드번호, API 키가 그대로 밖으로 나갈 수 있습니다. 일반 HTTP 프록시는 MCP의 stdio 통신을 보지 못해서, HTTP 리버스 프록시와 stdio JSON-RPC 어댑터를 Python으로 함께 만들었습니다.

주민등록번호는 체크섬과 날짜, 카드번호는 Luhn과 IIN, 계좌번호는 은행 문맥, API 키는 접두어와 엔트로피로 판별합니다. 탐지한 값은 정책에 따라 가리거나 막습니다. 검사 중 오류가 나면 통과시키지 않고(fail-closed), 감사 로그에는 원래 값을 남기지 않습니다.

명세	
기간	2026.07
구분	개인 프로젝트, 1인, 합성 데이터만 사용
역할	정책과 탐지 설계, 구현, 검증. 우회 사례와 테스트 초안은 AI 제안을 검토해 골랐습니다
기술	Python, HTTP 리버스 프록시, MCP stdio JSON-RPC, Prometheus, Docker, Helm
저장소	github.com/ki0915/Mcp-Guard

결과 우회 표현을 풀어서 검사하게 바꾼 뒤 합성 우회 표본 60건 중 탐지가 10건에서 56건으로 늘었고, 정상 표본 30건에서 오탐은 없었습니다. 로컬 측정에서 요청당 추가 지연은 평균 3.10ms였습니다.

호출하는 쪽과 외부 LLM 사이에 검사 경계를 하나 둡니다



배포 정책은 ConfigMap, 보호할 값은 Kubernetes Secret에 SHA-256으로만 보관. Docker와 Helm, non-root, 읽기 전용 루트 파일시스템, NetworkPolicy

저장소의 구조도를 이 문서에 맞게 다시 그렸습니다.

띄어 쓴 주민번호와 전각 숫자는 처음 버전에서 그냥 통과했습니다

처음 버전: 원문만 검사

`900101 - 1 2 3 4 5 6 7`	통과	숫자 사이 공백
`９００１０１－１２３４５６７`	통과	전각 숫자
`OTAwMTAxLTEyMzQ1Njc=`	통과	Base64

검사용 사본을 추가한 뒤 (가림은 원문 위치에)

NFKC 정규화	탐지	
공백, zero-width 문자 제거	탐지	
Base64 1회 해제	탐지	후보 64개, 64KiB 상한

형식만 보인 합성 값입니다. 실제 개인정보는 테스트에 쓰지 않았습니다.

지표	결과	조건
내장 회귀 테스트	35/35, 오탐 0/14	합성 한국 개인정보와 시크릿
우회 표본 탐지	**10/60 → 56/60**	원문만 검사할 때와 비교
정상 표본 오탐	0/30	고정 정상 표본 30건
HTTP 추가 지연	평균 3.10ms, p95 3.65ms	1,102B 요청 200회, 로컬
SSE 첫 이벤트	37.158 → 4.892ms	검사 모드 비교, 25ms 간격 이벤트 100회

아직 못 막는 것

우회 표본 4건은 여전히 놓칩니다. CI 테스트에 남겨 두고 추적합니다. 응답을 막아도 이미 나간 요청은 되돌릴 수 없고, 작은 합성 데이터셋 결과라 실서비스 탐지율을 뜻하지는 않습니다.

비싼 모델은 앞 단계에서 걸러진 소수의 후보에만 씁니다

1
정적 탐지
Python AST와 JS/TS 패턴으로 인증이나 검증을 하는 함수를 후보로 고릅니다.
LLM 토큰 0

2
분류
저비용 모델이 후보마다 가설과 신뢰도를 붙입니다.
저비용

3
멀티턴 검증
걸러진 소수만 고성능 모델로 여러 번 확인합니다.
고비용, 소수만

4
보안 영향 확인
기능 버그와 취약점을 가르고 OSV, GHSA와 중복을 봅니다.
저비용

5
SARIF 2.1.0
CWE, 심각도, 위치를 GitHub Security 탭에 올립니다.
CI 게이트

eval이나 exec 같은 위험 함수만 보지 않고, validate나 verify처럼 보안 판단을 하는 함수를 후보로 잡습니다. 권한 우회는 위험 함수 호출 없이도 생기기 때문입니다.

LLM 호출 비용은 모두 기록하고, 예산 상한에 닿으면 멈춥니다. PR과 push에는 토큰을 쓰지 않는 정적 모드만 돌리고, 심각도 high 결과가 나오면 GitHub Actions job이 실패합니다.

명세	
기간	2026.07
구분	개인 프로젝트, 1인
기술	Python AST, Claude API, SARIF 2.1.0, GitHub Actions, OSV, Docker
저장소	github.com/ki0915/etk-scanner

결과 SecurityEval 취약 코드 121건 중 111건을 찾았습니다. 이 도구를 활용해 분석한 오픈소스 인가 결함을 제보했고, 유지관리자에게 영향 확인을 받았습니다.

TypeScript 실습 백엔드에서 찾은 하드코딩 JWT 키로 토큰을 위조해 봤습니다

정적 단계 결과 (LLM 토큰 0)

```
[9.0] secretKey  login.controller.ts  hardcoded_secret  CWE-798
[9.0] secretKey  verify.token.ts      hardcoded_secret
[5.5] authMiddleware                  jwt.verify sink
```

실습 서버에서 수정 전후 비교

토큰 없음	수정 전 401	수정 후 401
유출 키로 위조한 토큰	수정 전 200	수정 후 401

CI 게이트

```
취약본  high 2  -> 실패
수정본  high 0  -> 통과
```

저장소 README에 남긴 실행 기록에서 옮겼습니다. 위조 토큰은 수정 전에는 200으로 통과했고, 수정 후에는 401로 막혔습니다.

지표	결과	읽는 법
SecurityEval 탐지	111/121	취약 코드 121건 기준
CWE 유형 일치	37.8%	탐지한 것 중 유형까지 맞은 비율
오탐률	측정 안 함	데이터셋에 안전한 표본이 없음

오픈소스 인가 결함 제보

이 도구로 한 오픈소스 프로젝트의 인가 처리 결함을 분석해 재현 자료와 원인 분석을 유지관리자에게 보냈습니다. 영향이 있다는 확인과 함께 고치겠다는 답을 받았습니다. 공개 전이라 프로젝트 이름과 세부 내용은 적지 않았습니다.

한국 AI 기본법 기준으로 AI가 한 일과 근거를 대조해 검토하게 돕습니다

실행 로그만으로는 AI가 무엇을 했고, 그때 권한이 있었는지, 어떤 근거를 검토했는지 설명하기 어렵습니다. AI 행동, 도구 결과, 당시 권한과 정책 버전, 원본 문서, 사람의 판단을 Ed25519 서명 원장으로 잇고, 확인하지 못한 사실은 unknown으로 남깁니다. 근거 문서는 암호화해 보관하고 버전이 바뀌면 재검토로 돌리며, 근거가 부족한 과제는 종결하지 않습니다.

AI 기본법과 시행령의 제30~36조, 제40조를 요구사항 18개로 옮겨 표시 의무, 고영향 판단, 위험관리와 사람 감독, 문서 5년 보관을 따로 검사합니다. 금융업이라는 이유만으로 고영향으로 정하지 않고, 법적 판단은 사람이 내립니다. AI 대출과 신용평가를 쓰는 금융사 합성 사례 6건에서 모두 기대한 판정이 나왔습니다. 요청마다 원장 전체를 다시 검증하던 병목도 고쳤습니다.

같은 1,000건 합성 시나리오	수정 전	수정 후
지속 구간 600건 중 접수	84	600
지속 구간 요청 p95	약 6초	206.81ms

명세	
기간	2026.09 – 2026.10
구분	개인 프로젝트, 로컬 파일럿 (AI 협업)
역할	문제 정의, 우선순위와 모델, 비용 조건 결정. 설계와 구현은 AI와 협업했습니다
기술	Node.js 24, SQLite WAL, Ed25519, HMAC, OIDC, Docker, Kubernetes
검증	금융 합성 사례 6/6
저장소	github.com/ki0915/EvidScope
설명	웹 포트폴리오에서 보기 ↗

다음 단계 다음은 실제 IdP와 SIEM 연동, 원격 KMS와 WORM 저장소 검증, 같은 감사 과제에서 근거를 찾는 시간 측정입니다. 실행을 막거나 법적 준수를 인증하는 도구는 아닙니다.

지난달 결과와 비교해 지적 항목이 조치됐는지 보는 월간 점검 스크립트입니다

KISA 주요정보통신기반시설 기술적 취약점 진단 가이드의 Unix 계정과 권한 10개 항목, 로그인 실패, 허용 목록 밖 포트를 점검하는 읽기 전용 스크립트입니다. 지난달 결과를 함께 넣으면 신규, 미조치, 조치 완료로 나누고 미조치 항목의 경과일을 셉니다. 중요도 상 항목이 실패하면 종료 코드 1을 내서 정기 작업의 경보로 이어지게 했습니다.

명세

기간	2026.10
구분	개인 랩 (AI 협업), 클라우드 Linux VM
역할	시나리오 기획, AI와 함께 구현, 직접 실행과 검토
기술	Python 표준 라이브러리, 단위 테스트 4건
설명	웹 포트폴리오에서 보기 ↗

```
양호 7 · 취약 3 · 해당 없음 2 ¦ 신규 0 · 미조치 3 · 조치 완료 2

U-02    패스워드 복잡성 설정      상  FAIL  미조치    31일
U-03    계정 잠금 임계값 설정     상  FAIL  미조치    31일
MON-02  허용 목록 밖 LISTEN 포트  중  FAIL  미조치    31일
U-47    패스워드 최대 사용기간    중  PASS  조치 완료
U-48    패스워드 최소 사용기간    중  PASS  조치 완료
U-04    패스워드 파일 보호        상  PASS  정상

조치 요청
U-02    pwquality.conf minlen>=8, minclass>=3
U-03    faillock.conf deny=5 unlock_time=600
```

랩 VM의 실행 결과 파일에서 옮겼습니다. 두 번째 실행은 날짜 인자로 한 달 뒤를 가정해 같은 VM에서 돌렸습니다.

1GiB 디스크에서 정리 없이 배포를 14번 하면 절반이 실패했습니다

시나리오 (1GiB 디스크, 배포 14회)	배포 성공	최대 사용률	쓰기 실패
정리 없음	7/14	100%	4회
배포마다 단계형 정리	**14/14**	91%	0회
dry-run	삭제 계획만 출력, 이미지 수 8 → 8		

정리 규칙

사용 중인 이미지, 보존 라벨이 붙은 이미지, 막 만든 이미지는 지우지 않습니다. 사용률이 70%를 넘으면 오래된 태그를, 85%를 넘으면 중지 컨테이너와 빌드 캐시를 정리합니다. 컨테이너 로그는 사고 때 증거가 되므로 지우지 않고 경보만 냅니다. 기본은 dry-run이고, 정리 내역은 JSON 감사 로그로 남깁니다.

실험하면서 틀렸던 것

첫 구현에서는 오래된 이미지를 지워도 사용률이 그대로였습니다. BuildKit 빌드 캐시가 같은 레이어를 붙잡고 있었고, 이미지 삭제 뒤 캐시 상한을 적용하자 91%에서 38%로 내려갔습니다.

정리를 배포 전에 돌리면 최고점이 낮아질 거라는 예상도 틀렸습니다. 배포 전이든 뒤든 최고점은 91%였고, 지켜야 하는 이미지들의 크기가 그 값을 정했습니다.

개인 랩 (AI 협업). 실무 당시의 디스크 용량과 사용률은 기록이 없어서, 이 수치는 모두 랩에서 잰 것입니다.

공격 경로는 실습 환경에서 따라가 봤고, 컨테이너 보안 기준은 직접 정리했습니다

TryHackMe 모의해킹 실습

Nmap으로 서비스를 열거하고, SMB 익명 접근과 IIS 공유 디렉터리 설정 오류가 어떻게 이어지는지 확인한 뒤 권한 상승까지 따라갔습니다. 원인, 영향, 개선 방안을 보고서로 정리했습니다.

명세	
구분	실습, 허가된 환경
기술	Nmap, SMB, IIS, 권한 상승, 보고서 작성

Docker 컨테이너 보안 가이드

OWASP Docker Top 10, CIS Docker Benchmark, NIST SP 800-190을 위험도와 구현 비용 기준으로 다시 묶어, 작은 팀이 먼저 할 것과 나중에 할 것을 나눴습니다. 이미지 최소화와 digest 고정, CI의 Trivy 검사, non-root 실행, BuildKit 시크릿, 네트워크 격리, Falco 런타임 감시를 다룹니다.

명세	
구분	저술, 공개 저장소
저장소	github.com/ki0915/Docker-Security-Guide

이메일	전화	GitHub	웹 포트폴리오	영문 이름
kiminsoo365@gmail.com	010-4678-3603	github.com/ki0915	ki0915.github.io/Profile/docs	Ki Minsoo